

National Tsing Hua University
Department of Electrical Engineering
EE429200 IC Design Laboratory, Fall 2025

Homework Assignment #4 (15 %, total 100 points)

QR Code Decoder

Assigned on Oct 23, 2025

Due by **Nov 6, 2025**

Assignment Description

I. Introduction



Fig. 1 Example of QR Code (14-byte “www.google.com”)

QR Code (Quick Response Code) [1] is a type of matrix barcode used for storing text information in a robust way. It is widely used for URL instant access. Its specification varies a lot, including version 1 to 40 (from 21x21 to 177x177 pixel), four different character sets, and different levels of error correction capability. In this assignment, you will implement the smallest version : version-1 (21x21 pixels, total 26 codewords), 8-bit Byte mode (ISO 8859-1 code), and error correction level L (7% recovery capacity) QR Code decoder.

II. Goal

The structure of QR Code is shown in Fig. 2. The version 1-L QR Code is a 21x21 binary matrix, in which the black pixel stands for “1”, and white for “0”. The codewords of a QR Code are “masked” (XORed) with one pre-defined pattern for balancing the number of black and white pixels. As a result, you need to find the mask pattern ID for de-masking the codewords. After de-masking, the codewords can be decoded sequentially. There are total 26 codewords. 19 of them are data codewords, and the other 7 of them are error correction codewords. Since the QR Codes in this assignment are **all error-free, error correction is not required to perform**. You can directly decode the text from the data codewords. All the decoding steps will be explained in detail in the next section.

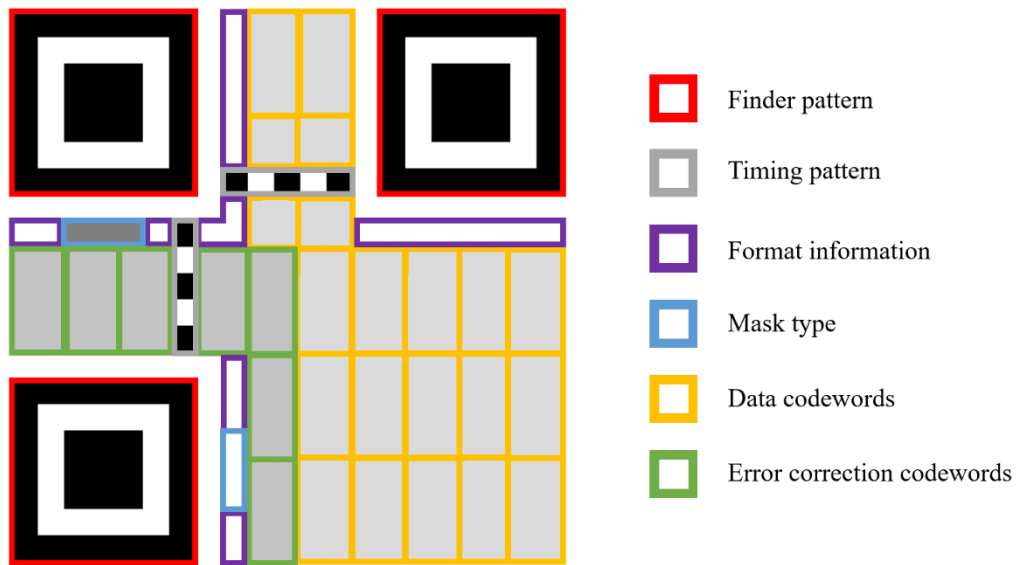


Fig. 2 Structure of version 1-L QR Code.

In this assignment, four different types of test pattern, Rank A, Rank B, Rank C, and Rank D, will be used to test your design. Rank A stands for the most complex pattern, and Rank D stands for the simplest pattern. Table 1 describes the format of test pattern for each Rank, and Fig. 3 provides some examples of test pattern for each Rank.

Pattern	# of QR Code in a pattern	Location	Rotation	Scale	Background
Rank A	Arbitrary (1 to 8)	Arbitrary	4 way	1x / 2x	Yes
Rank B	Arbitrary (1 to 8)	Arbitrary	4 way	1x	Yes
Rank C	1	Arbitrary	4 way	1x	No
Rank D	1	Upper-Left	0-degree	1x	No

Table. 1 Format of test pattern for each Rank

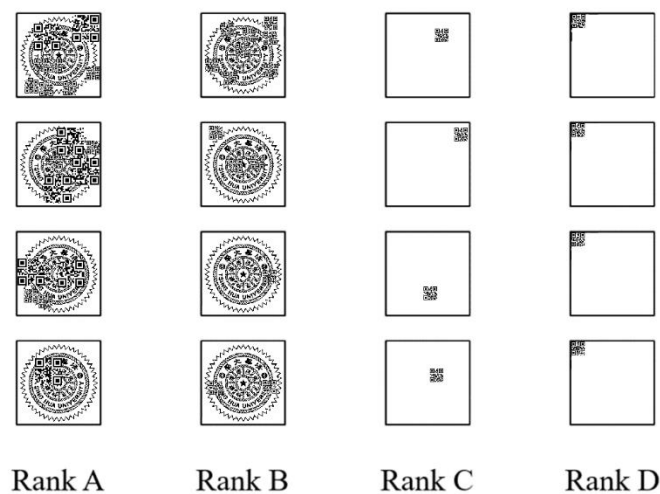


Fig. 3 Examples of test pattern for each Rank

III. Detailed description of decoding a QR Code

1. Data arrangement in SRAM

The 21x21 (scale:1x) or 42x42 (scale:2x) QR code patterns are put into an 128x128 image, and the whole 128x128 image is stored in SRAM. The data arrangement is illustrated in Fig. 4. The SRAM has a capacity of 1024 words and a 16-bit word size. Each address contains 16 pixels, arranged in the order shown in Fig. 4.

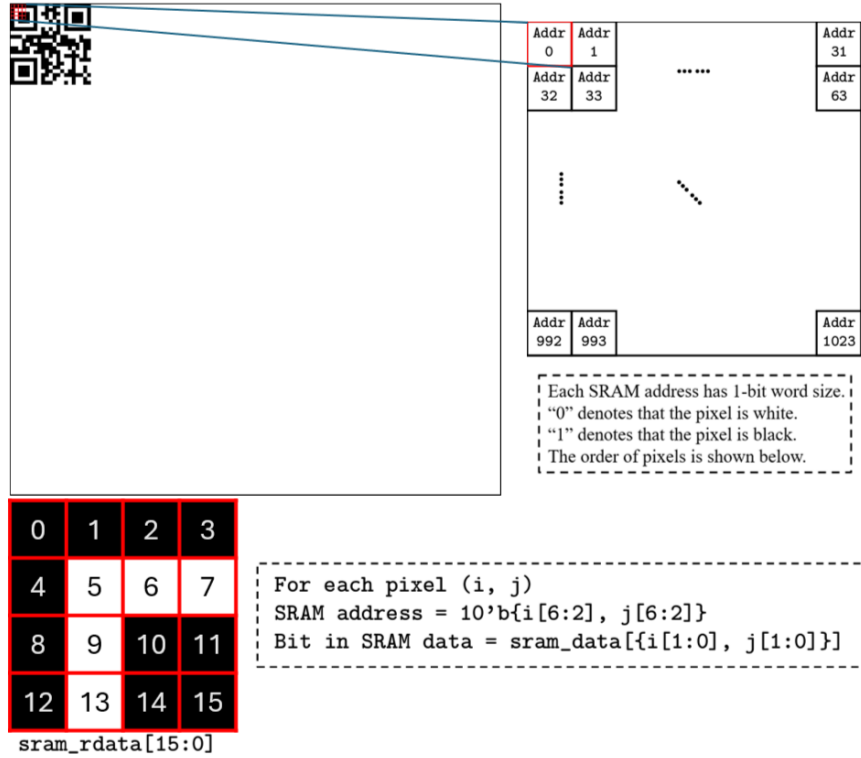


Fig. 4 Data arrangement in SRAM.

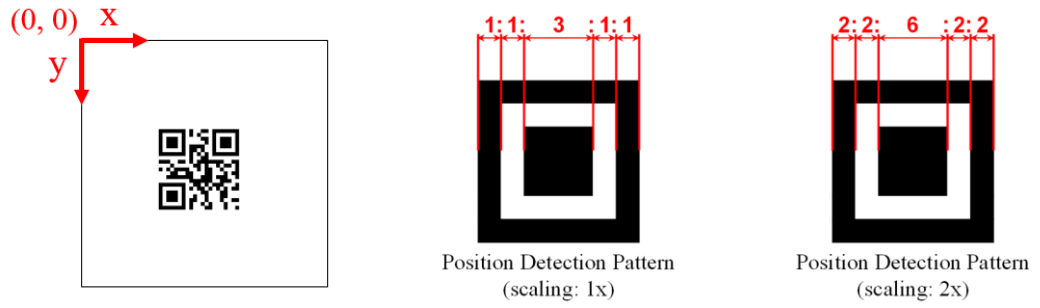
2. Rotation of the QR Code

For Rank A, Rank B and Rank C, the QR Codes are at arbitrary location with arbitrary rotation ($0^\circ, 90^\circ, 180^\circ, 270^\circ$). You need to locate the QR Codes and determine their rotation before decoding it. The rotation of the QR Codes depend on the location of the three finder patterns. Fig. 5 illustrates the definition of all four kinds of rotation.

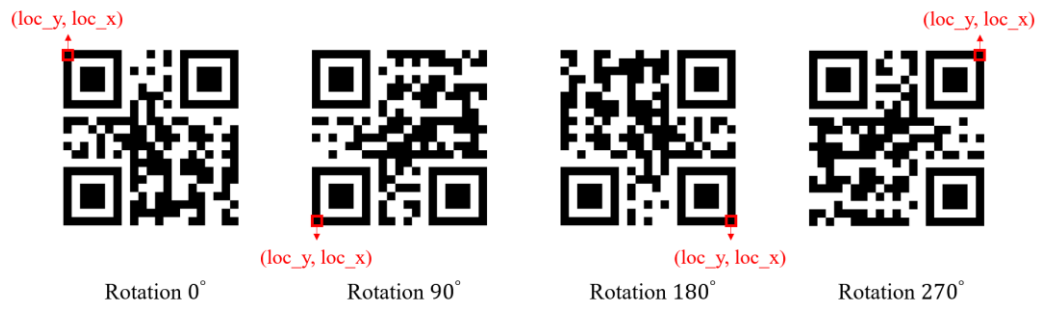
3. Location of the QR Code

For Rank A and Rank B, the number of QR Code in a test pattern might be more than one. Therefore, we use two output ports: $[6:0] loc_y$ and $[6:0] loc_x$ to specify which QR Code in the test pattern does the output $[7:0] decode_text$ belongs to. The definition of the QR Code location is illustrated in Fig. 5.

Note: Although there is only one QR Code in each pattern of Rank C / D, you should still specify and output the location of the QR Code through loc_y and loc_x .



Scale: 1x



Scale: 2x

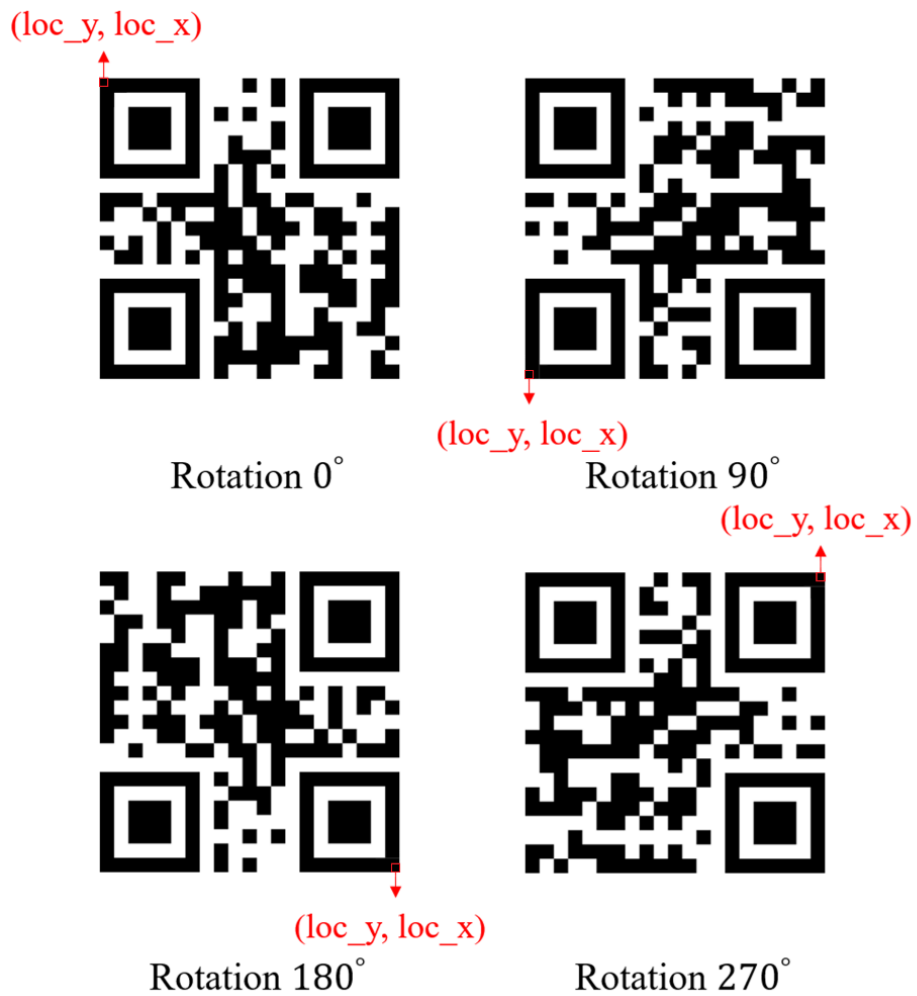


Fig. 5 Definition of location and rotation of the QR Code

4. De-masking process

All the codewords in the QR Code are masked (XORed) with a specific mask pattern, and the mask pattern is decided by the mask pattern ID. The ID is a 3-bit data, which locates in position $\{(8, 2), (8, 3), (8, 4)\}$ in the QR Code with rotation 0° . **The 3-bit data should first be XORed with a fixed 3'b101 to derive the real mask ID.** There are total 8 different mask patterns. The pixel of the mask pattern is equal to "1" if it satisfies the corresponding condition in Fig. 6. You can refer to Fig. 7 for what each mask pattern should look like. An example of the de-masking process is provided in Fig. 8.

Mask Pattern Reference	Condition
000	$(i + j) \bmod 2 = 0$
001	$i \bmod 2 = 0$
010	$j \bmod 3 = 0$
011	$(i + j) \bmod 3 = 0$
100	$((i \div 2) + (j \div 3)) \bmod 2 = 0$
101	$(ij) \bmod 2 + (ij) \bmod 3 = 0$
110	$((ij) \bmod 2 + (ij) \bmod 3) \bmod 2 = 0$
111	$((ij) \bmod 3 + (i+j) \bmod 2) \bmod 2 = 0$

Fig. 6 Mask generation condition [1]

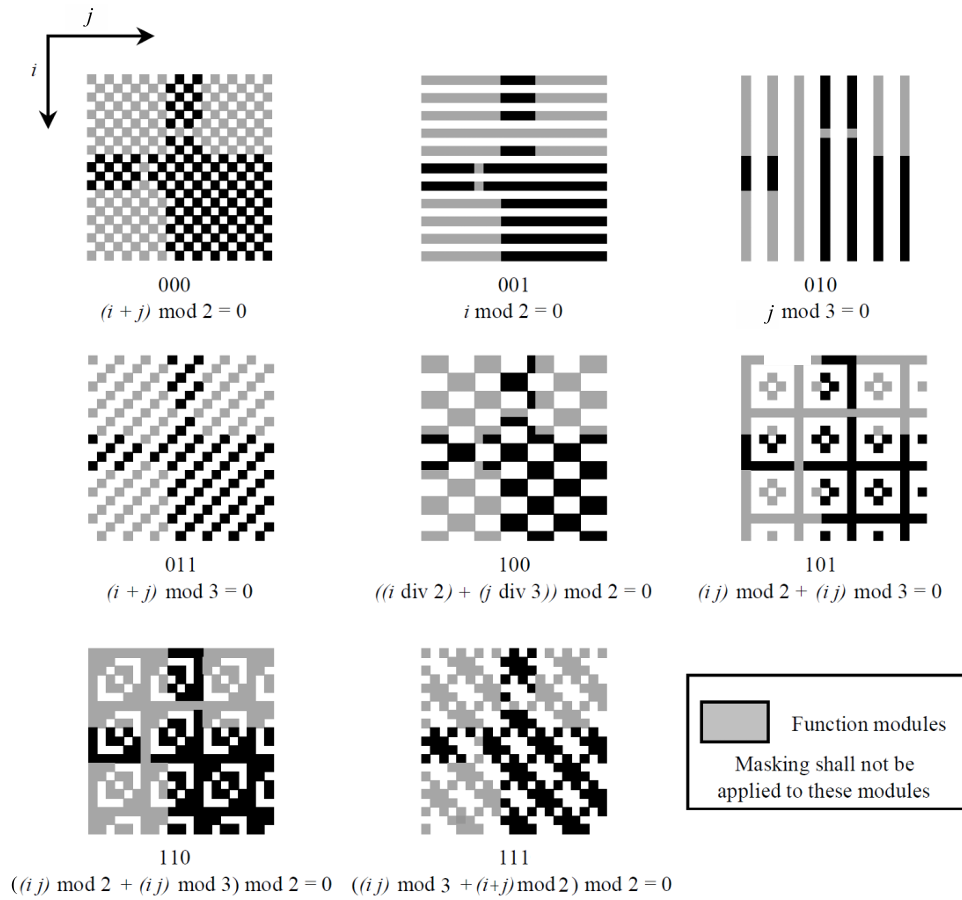


Fig. 7 Eight kinds of mask pattern [1]

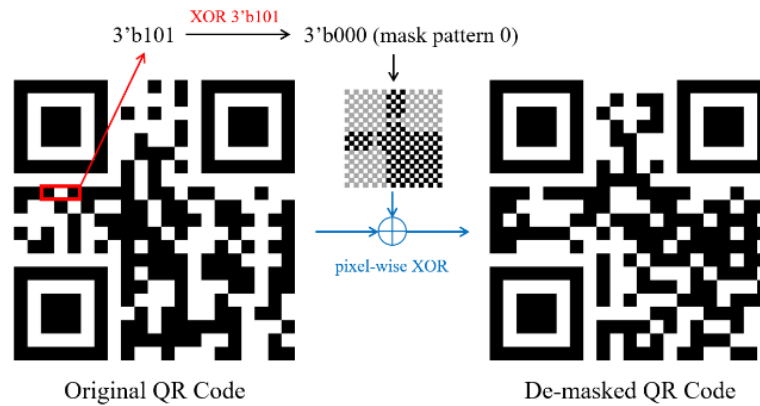


Fig. 8 Example of de-masking process

5. Decode the data codewords

After demasking, you can read data codewords from the de-masked QR Code. The arrangement of each data codeword and its bit order is shown in Fig. 9. Decode the codewords by the order of number labelled in each block.

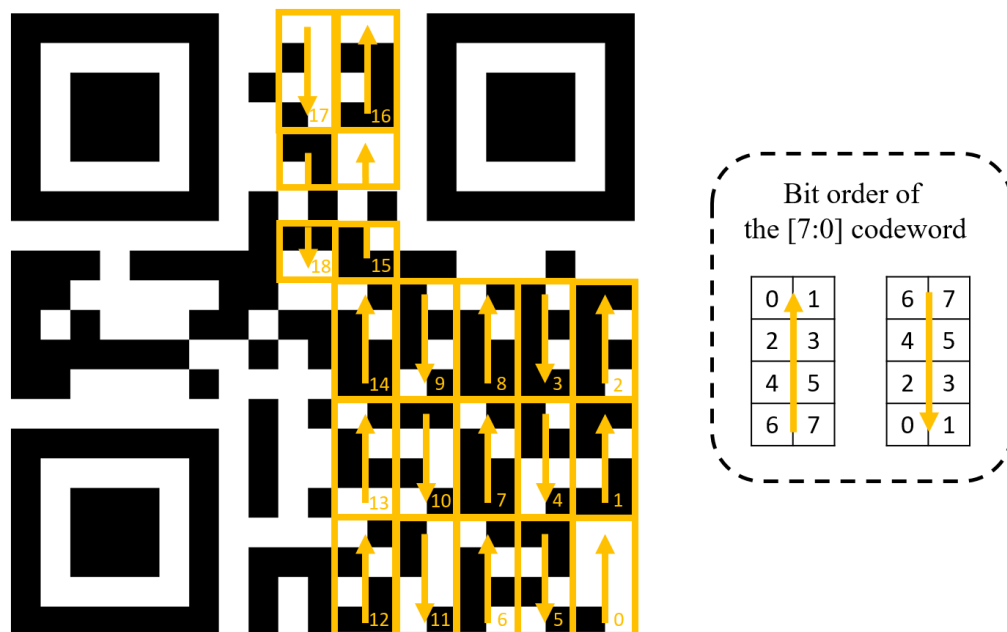


Fig. 9 Arrangement of each data codeword and its bit order

6. Decode the text sequentially

The data in data codewords is arranged as shown in Fig. 10. The first four bits in codeword 0 is data encoding type. We only use 8-bit Byte mode (4'b0100) in this assignment, you can refer to ISO 8859-1 code to see the value of each character. The last four bits of codeword 0 and the first four bits of codeword 1 indicates the text length encoded in this QR Code. You should first determine the text length to decide how many characters should be decoded. For the rest of the

data codewords, the last 4 bits of the current codeword and the first four bits of the next codeword will form a text data. Once decoding a text data, you can put it to the output port $[7:0]$ *decode_text* and set the output port *valid* to high. The testbench will check the output text when the output valid is high.

Codeword 0	Codeword 1	Codeword 2	Codeword 3
0 1 0 0 0 0 0 0	1 1 1 1 0 1 1 0	0 0 0 1 0 1 1 0	0 0 1 0 0 1 1 1
Data encoding type (4'b0100)	Text length (8'b00001111)	Text 0 (8'b01100001)	Text 1 (8'b01100010)

Fig. 10 Data arrangement in data codewords

I/O Definition

Type	Name	Number of bits	Description
Input	clk	1	Clock
Input	srst_n	1	System reset (synchronous, active low)
Input	start	1	Indicate that you can start decoding (one-cycle pulse)
Input	sram_rdata	16	SRAM read data
Output	sram_raddr	10	SRAM read address
Output	loc_y	7	Row location of the QR Code
Output	loc_x	7	Column location of the QR Code
Output	decode_text	8	Decoded text
Output	valid	1	Indicate that the decode_text is valid when it is high
Output	finish	1	Set high after all QR Codes are decoded

Timing Diagram

Take a test pattern which contains two QR Codes as an example. One QR Code is located at (0, 0), and its decoded text is “abc” which the number is {8'h61, 8'h62, 8'h63} respectively in ISO 8859-1 code. The other QR Code is located at (30, 40), and its decoded text is “NTHU” which the number is {8'h4e, 8'h54, 8'h48, 8'h55} respectively in ISO 8859-1 code.

Fig. 11 shows the timing diagram for this example. Once the one-pulse *start* signal is detected, the circuit can start to operate. When one QR Code in the test pattern is decoded, the decoded text of the QR Code and its location is output sequentially, and the output *valid* is set high to indicate that the output *decode_text* is valid. After that, when the other QR Code is decoded, the decoded result is output in the same way. After all the QR Codes in the test pattern have been found and decoded, set the output *finish* high to tell that the work is done.

When there are more than one QR Code in the test pattern, your design should be able to find and decode all of them. But **the output order of these QR Codes is not limited**. In the above example, you can output the QR Code at (0, 0) first as shown in Fig. 11, or you can output the QR Code at (30, 40) first as shown in Fig. 12.

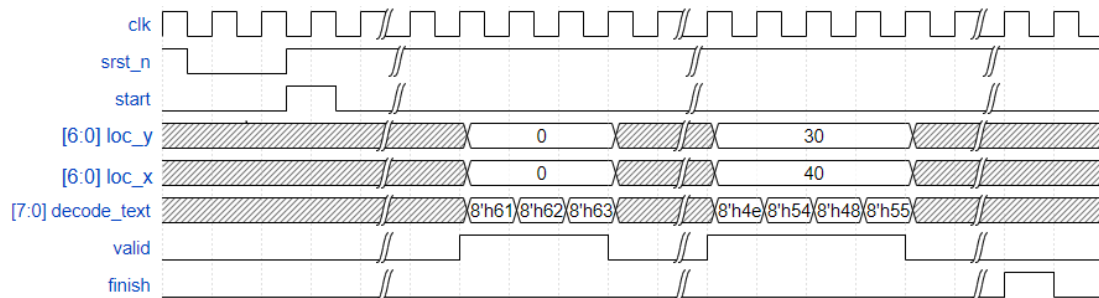


Fig. 11 Timing diagram of the test pattern which contains two QR Codes
(QR Code at (0, 0) is output first)

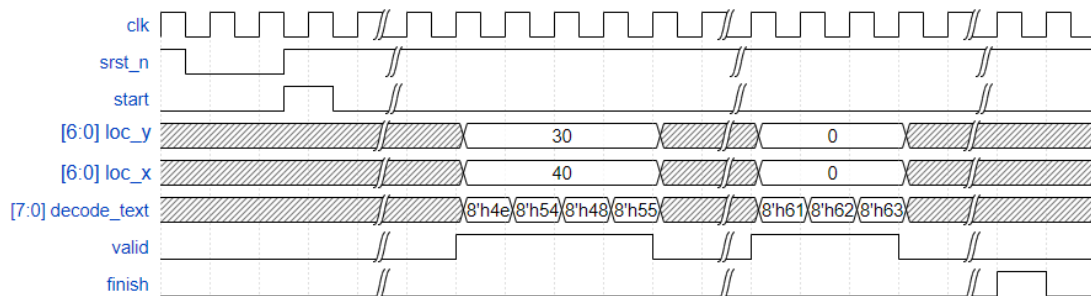


Fig. 12 Timing diagram of the test pattern which contains two QR Codes
(QR Code at (30, 40) is output first)

Besides, when output the decoded result of a QR Code, the output *decode_text* is not mandatory to be consecutive. Fig. 13 provides an example of nonconsecutive output behavior. However, the order of the output *decode_text* should still be correct.

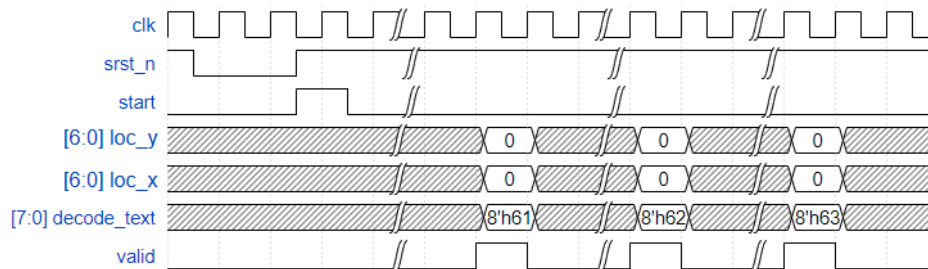


Fig. 13 Another possible timing diagram when decoding a QR Code

Notice that although the output order of the QR Codes in a test pattern is not limited, the output behavior like Fig. 14 is not acceptable. You can only output the decoded result of the next QR Code after all the decoded result of the previous one have been output.

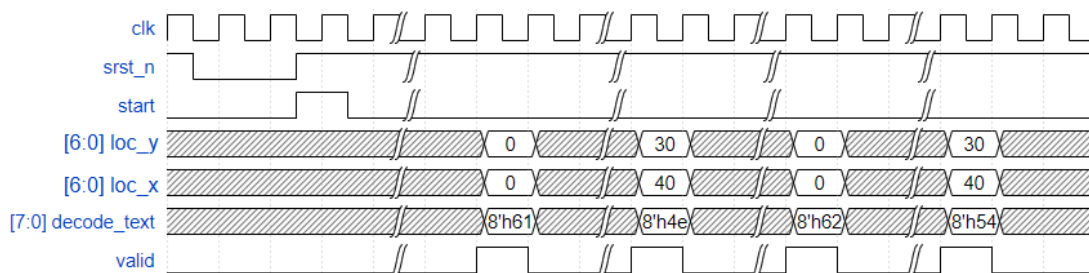


Fig. 14 Not acceptable output behavior

Grading Policy

The grading policy is divided into two part: functionality score and synthesis / PI score. The functionality score depends on which types of test pattern (Rank A, Rank B, Rank C, Rank D) your design could pass during the pre-simulation. The synthesis PI score depends on the synthesis performance of your design.

1. Functionality (60 points)

Note that your design (RTL) must pass spyglass (synthesizable).

Rank A (20 points): multi-QRcodes with noisy background, random position / rotation / scale (1x or 2x).

Rank B (15 points) : multi-QRcode with noisy background, random position / rotation.

Rank C (15 points) : one QRcode with clean background, random position / rotation.

Rank D (10 points) : one QRcode with fixed position and clean background.

2. Synthesis (0 points)

For fairness, please use the provided script to synthesize your design. Only two script files can be modified: **0_readfile.tcl** for adding your self-defined RTL files, and **synthesis.tcl** for changing the clock timing constraint TEST_CYCLE (e.g. 5.0 for 5.0 ns). **Note that your timing slack should not be negative.**

3. PI (40 points)

Only for those who pass simulation of all ranks.

A. (5 points)

You will receive 5 points if you pass simulation of all rank and **correctly fill in your PI in the ranking list.**

B. (35 points) PI Ranking

You can earn up to 35 points based on your PI ranking among all eligible students. Only students who pass all ranks and satisfy **$PI \leq 6.00 \times 10^9$** are eligible.

The performance index (PI) for this assignment is defined as:

$$PI = Area \times Timing \times Cycle$$

Where:

Area: Total cell area shown in report_area_qrcode_decoder.out, E.g. 2000um².

Timing: Clock period of the timing constraint when synthesis. E.g., 5ns

Cycle: Total cycle count to complete the **whole simulation of Rank A**. It is shown on the terminal such as below when the whole simulation is passed and finished:

```
===== Summary of Rank A=====
Congratulation! All patterns are successfully passed! \(\O v \O)/
Total cycle count C = xxxxxx
```

The score is computed using a Gaussian-based ranking formula:

$$\text{Score} = 35\% \times (\mu + \sigma \times \Phi^{-1} \left(1 - \frac{Rank-0.5}{N} \right)) \text{ points}$$

Where:

Rank = your rank (1 for the best performer)

N = number of **all students in ICLAB**, which is 76

$\Phi^{-1}(\cdot)$ = inverse of the standard normal cumulative distribution function

μ and σ = normalization parameters to map ranks into score range, $\mu = 60$, $\sigma = 20$.

C. Additional 2% and 1% of the total score of the semester are given to the students with the lowest and second lowest PI

Simulation

For Rank A, there are total 300 patterns.

For Rank B, Rank C, and Rank D, there are total 200, 180, 60 patterns respectively.

To pass each Rank, your design should pass all patterns of the Rank.

Simulation commands for each Rank are as below:

```
# Rank A
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_A+DEBUG_MODE+PAT_L=0+PAT_U=299

# Rank B
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_B+DEBUG_MODE+PAT_L=0+PAT_U=199

# Rank C
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_C+DEBUG_MODE+PAT_L=0+PAT_U=179

# Rank D
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_D+DEBUG_MODE+PAT_L=0+PAT_U=59
```

For debugging convenience, you can define **DEBUG_MODE** and modify **PAT_L** and **PAT_U** to simulate with any specific range of patterns. For example, to simulate pattern No. 0 to No. 3, you can assign **PAT_L** to 0 and **PAT_U** to 3.

```
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_D+DEBUG_MODE+PAT_L=0+PAT_U=3
```

If you just want to simulate with one certain pattern, you can assign **PAT_L** and **PAT_U** to the same number. For example, to only simulate pattern No. 50, you can assign both **PAT_L** and **PAT_U** to 50.

```
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_D+DEBUG_MODE+PAT_L=50+PAT_U=50
```

Deliverable

1. Synthesizable verilog

- qrcode_decoder.v
- All other your own defined RTL files (*.v) (if any)

2. Command-line arguments file

- hdl.f (should include all the RTL files used in your design)

3. Spyglass report

- spyglass.rpt

4. Synthesis scripts and results

- 0_readfile.tcl
- synthesis.tcl

- da.log
- report_time_qrcode_decoder.out
- report_area_qrcode_decoder.out
- qrcode_decoder_syn.v
- qrcode_decoder_syn.sdf

5. Text file

- misc.txt

Summarize your Grade and PI, including each value of A, T, and C, of your submitted design. A template is provided in the released package.

File Organization

Directory	Filename
HW4_{studentID}/	misc.txt
	spyglass.rpt
HW4_{studentID}/hdl/	qrcode_decoder.v
	All other your own defined RTL files (*.v) (if any)
HW4_{studentID}/sim/	hdl.f
HW4_{studentID}/syn/	0_readfile.tcl
	synthesis.tcl
	da.log
	report_time_qrcode_decoder.out
	report_area_qrcode_decoder.out
	qrcode_decoder_syn.v
	qrcode_decoder_syn.sdf

Note:

If your studentID is 123456789, HW4_{studentID} denotes HW4_123456789.

Compress your whole folder HW4_{studentID} into HW4_{studentID}.zip and submit to eeclass.

Wrong file delivery or wrong file organization will get 1 point punishment.

Reference

- [1] ISO/IEC 18004:2000(E), *Information technology — Automatic identification and data capture techniques — Bar code symbology — QR Code*.